Dashboard — What we expect at the defense

G3 Full-Stack Web Development · read it with your teammate

This is what we expect to see. How you do it is your decision. Tick a line only when you have tested it yourselves.

Starting the project	
[ ]	Your project builds and runs with `docker-compose`, from a fresh clone, on any machine.
[ ]	Nothing to do by hand before, during or after.
[ ]	Your server answers on port **8080**.
[ ]	No error while it starts.
about.json	
[ ]	`/about.json` is available and returns valid JSON.
[ ]	It gives the client host, the server time, and the full description of your services, widgets and parameters.
[ ]	It always matches your application.
Accounts	
[ ]	Registration with an email address.
[ ]	The account requires a verification.
[ ]	Login with a password.
[ ]	Passwords and tokens are never readable by anybody.
[ ]	External accounts are connected with OAuth 2.0, for the user who is logged in.
Services and widgets	
[ ]	At least **5 services** and **8 widgets**. Do not limit yourself to the minimum.
[ ]	At least one service usable without a subscription.
[ ]	Every widget is configurable by the user, and its parameters are typed.
[ ]	The same widget, used twice with different settings, gives different results at the same time.
The dashboard	
[ ]	The user chooses a widget, configures it, sets its refresh rate and confirms it.
[ ]	The user can reconfigure, move and delete a widget.
[ ]	The dashboard belongs to the user and survives a reload.
[ ]	Each widget refreshes at its own rate.
Security	
[ ]	Your data is checked on the server side.
[ ]	Your variables and your secrets are protected.
[ ]	Nothing sensitive is readable anywhere, including in your git history.
[ ]	Your repository is clearly organised, with no duplicated code.
Interface	
[ ]	A consistent visual identity.
[ ]	The user always knows what is happening: loading, errors, empty screens.
[ ]	Usable on any screen size.

[ ]	Accessibility is taken into account.
Technology choices	
[ ]	For each layer of your project, compare at least **3 technologies** before choosing.
[ ]	A POC in each of them. It can be very short.
[ ]	A written comparison with your final choice and the reason for it. It can be short too.
[ ]	Say what each of you already knew, and how it helped you decide.
README	
[ ]	Someone who has never met you can install, run and understand your project with it.
[ ]	Make a real effort in English. The quality of the language counts.
Prepare the defense	
[ ]	Understand your project globally, not only the part you wrote.
[ ]	Be ready to explain the logic, not to read your code.
[ ]	Know how the data goes from the frontend to the backend and comes back.
[ ]	Know where you validate the data, and which HTTP codes you return and why.
[ ]	Both of you can answer on any part of the project.
[ ]	Come with the project already running.

NB

- If you need help, ask. We are here for that, and we would rather help you now than watch you fail later.
- At the defense we will be strict. A functionality that does not work is a functionality that does not exist. We will not accept excuses.
- **Any logic you cannot explain is not validated, even if it works on the screen.**
- You are validating skills. We will not let a student claim a skill they cannot explain.
- You implemented a widget, you explain how. You integrated a service, you know how you got the client ID and the secret key.
- You explain in English. So prepare it, and practise before the defense.
- Do not be surprised to have every functionality working and still fail.

“If you can’t explain it simply, you don’t understand it well enough.”
often attributed to Albert Einstein